

Pending Template Updates — Design

The core problem

Users need to see which of their engagements have pending template updates, and what those updates change. The difficulty is that the template version an engagement is on is stored only inside the engagement file, and reading it means loading the file (~1 minute, a hard constraint). The template store, shared by all firms, knows nothing about engagements.

So the design keeps a small table per firm recording which template and version each engagement is on. A one-time background job (the *backfill*) fills it by loading every existing engagement once; code added to the engagement management system keeps it current from then on. Every user-facing question is answered from that table and the list of published versions, so no engagement is ever loaded while a user waits.

Key assumptions

1. Listing a firm's engagements (id, name) is cheap; only reading template/version inside them is slow.
2. An engagement can be loaded just to read its template ID and version without changing the client's file (still ~1 min); it carries a *revision number* that increases on every save.
3. An engagement's template version changes only when it is created or an update is applied; both go through the engagement management system (as do declines), so updating our tables from inside those actions keeps them correct.
4. Tables can be added to each firm's existing database and the engagement management system can write them in the same transaction as the engagement change (if not, see *Plan B* in §5); firm-scoped credentials can be issued.
5. Versions are numbered in order and are cumulative (v5 contains v4's changes); any intermediate version can be applied; the template store can list each template's versions. A published version is never withdrawn: a correction ships as a new version.
6. Closed engagements from prior years do not take template updates.
7. "Up-to-date" means correct whenever the user looks; push notifications are an optional extension.

1. Target Architecture

How it works for the user

- **At a glance.** Each engagement in the list shows *Up to date*, *N updates pending* or *Verifying...* (its version is not known yet; the engagement itself stays fully usable).
- **What counts as pending.** Any published version of the engagement's template that is newer than the version it is on and newer than anything the user already declined.
- **Accumulated versions, one review.** The user applies everything up to the latest version in one step. If they decline the latest, they are offered the one before it, so they can take v4 and skip v5 (versions build on each other: v4 can be taken without v5, not the reverse). A declined version is not offered again.
- **Summaries.** One per version, shown in order: a user on v3 reads what v4 changed, then what v5 changed. Items changed in more than one of them are marked *changed again*. Summaries are labelled *AI-generated* and never recommend apply or decline. Until a summary is ready the update shows as pending with *Summary in preparation*, and the review opens once it is.
- **History.** Every apply/decline is recorded: who, when, from which version, to which.

Example: an engagement on v3; v4 and v5 are published → *2 updates pending*. The user declines v5 and applies v4 → the engagement is on v4 and v5 is not offered again. v6 ships → *1 update pending*; because v6 builds on v5, the review shows v5's summary too, marked *previously declined*, followed by v6's.

How it is built

Component	What it does	On AWS
Queue	Holds each "new version published" message from the template store until processed; retries on failure	EventBridge rule → SQS
Release processor	Records each new version and requests its summary	Lambda
Summary generator	Writes the human-readable summary of each new version (see below)	Lambda + Bedrock (LLM)
Engagement hook	Code added to the engagement management system's create / open / apply / decline / archive actions; updates our tables in the same transaction. Opening already loads the engagement, so its version is recorded at no extra cost	New code inside the existing system (no new AWS service)
Backfill job	Loads each existing engagement once (~1 min) to record its template and version	ECS tasks per region
Pending Updates API	Serves the engagement list by comparing version numbers	API Gateway + Lambda, each region
Monitoring jobs	<i>Version check:</i> compares our list of versions with the template store's and adds any missing. <i>Spot check:</i> opens a few random engagements per firm in the background and compares their real version with our table. <i>Impact count:</i> after each publish, counts per firm the engagements it leaves pending	Scheduled Lambdas + CloudWatch

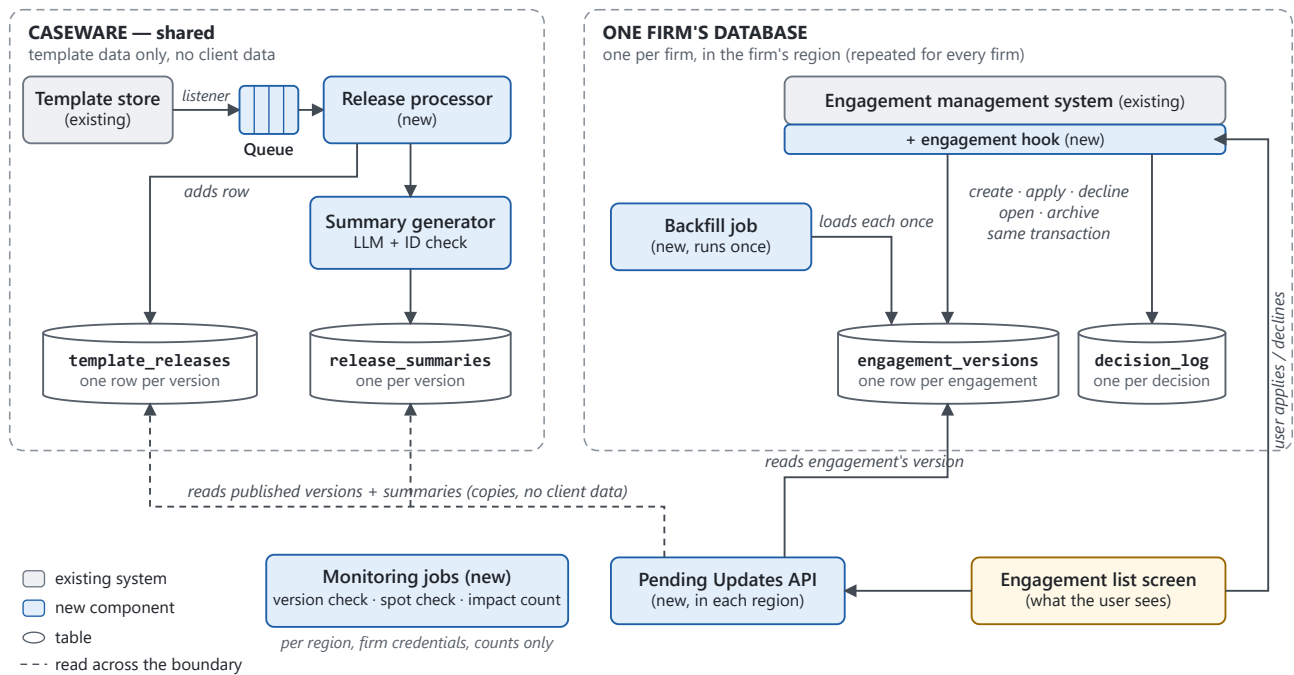


Table	Stores	Lives in
engagement_versions	Which template and version each engagement is on, and what the user has declined	Each firm's database
decision_log	Every apply/decline decision (append-only)	Each firm's database
template_releases	Every published version of every template	Shared, copied to each region (no client data; e.g. DynamoDB global table)
release_summaries	Each version's summary and review status; a correction adds a dated revision, never overwrites	Same as above

Summary generation (at publish time). (1) The diff tool compares the new version with the previous one; each change gets an ID (C1, C2...). (2) The LLM receives the changes and, as context, only the parts of the previous version they touch, so it can name things ("procedure 12" is "Lease review"); the whole template would add cost and noise. (3) It writes the summary citing the change IDs. (4) *ID check* (code): every change is cited and no cited ID is invented; otherwise one retry, then human review. *Changed again* is computed by code from the paths each version changed. Rules per change type were rejected: we do not know the template's internal structure.

Security. engagement_versions and decision_log reveal a firm's clients and activity, so they live in each firm's existing database and inherit its isolation and data residency. Backfill and monitoring use firm-scoped credentials; the API takes the firm from the session token, never from the request, and reuses existing per-engagement permissions; logs carry IDs only; everything that touches firm data runs in the firm's region. Only template data is shared, and the LLM only ever sees template content, never engagement or client data; monitoring exports counts only.

2. Implementation Plan

Each phase has a per-firm on/off switch; we only add tables and code and never modify an engagement file, so rollback is switching it off.

- Tables.** Firm tables in each firm's database (a migration per firm); shared tables once. *Done when* migrated for the pilot firms.
- Capture new changes.** Enable the engagement hook and the template-store listener; copy the versions already published into template_releases once. This goes **before** the backfill, so changes made while it runs are not missed. *Done when* pilot firms show hook writes and the version check finds nothing missing.
- Backfill.** Works through each firm's *verifying* engagements, recently opened first; loads each without changing it, records template, version and revision number, marks it verified (skipping any the hook already recorded). After 3 failed loads: **error** and an alarm. Mostly off-hours, a limited number at a time, pausing if users' load times rise. Size (assumed): ~150,000 engagements ≈ 3 nights at 100 at a time. *Done when* a firm's open engagements are verified or in **error**.
- API, hidden.** Build the Pending Updates API and deploy it in each region with the UI off; the spot check confirms its answers.
- Turn it on.** Pilot firms first, then the rest in groups.
- Summaries (in parallel with 3–5).** Evaluate on past versions; launch with the content team approving every summary; move to sampled review once they are consistently approved without edits.

3. Testing Strategy

Component	What we test	Passes when
Pending rule (built, implementation/)	The \$1 example; walk-back and re-offers; stale backfill readings; versions published mid-review; 500 random histories	32 tests pass: a declined version is never re-offered, none is declined unseen, an engagement never moves back
Pending Updates API	Firm taken from the session	No engagement is ever loaded; firm A never sees firm B

Component	What we test	Passes when
Engagement hook	Create / apply / decline, including a failing one	Our tables change only if the engagement change succeeds
Backfill job	A copy of a pilot firm, with forced load failures and a user applying mid-run	Every engagement ends verified or in <code>error</code> ; a newer apply is never overwritten
Queue + release processor	The same publish message twice, or out of order	One row per version
Summary generator	ID check, plus past versions graded by the content team	No invented or missing change; a prompt or model change ships only if the grade holds

4. Evaluation & Observability

For each component we measure whether it does its job correctly, not only whether it is running. Thresholds are set in the pilot.

Component	Doing its job right means	How we measure it in production
Queue + release processor	No published version is missing	Version check: missing versions must be zero
Hook, backfill and API	Every engagement shows exactly the updates it really has pending	Spot check: mismatches must be zero; backfill progress per firm; list response time flat with firm size
<code>decision_log</code>	Every decision is recorded	Every version change has a log entry: each firm's decision history
Impact count	Answers how many engagements a new version affects	Per firm and region, numbers only: engagements pending or still verifying. A firm missing from the count, or with no spot check result, means the check is not running for that firm

Evaluating the summaries. Good means *complete, faithful* (nothing invented, meaning right), *clear* to a non-technical auditor and *neutral*; the ID check proves only completeness. **Before release:** the graded set of past versions gates every prompt or model change. **In production:** at about one summary per product per week, the content team grades them (all at launch, then a sample); we track approvals without edits and edit reasons. **Users:** an "accurate?" control and a "wrong summary" flag, confirmed by the content team; alarm on any confirmed error. **Is the feature useful:** time from publish to decision, and engagements pending for months.

5. Failure Modes & Tradeoffs

Failure → effect	Detected by	Mitigation
Publish message lost → wrongly <i>Up to date</i>	Version check	Queue retries; the version check adds the missing version
Summary wrong or incomplete → bad decision	ID check (invented, omitted); content-team grading and user flags (wrong meaning)	Approved at launch; fixed once for every firm; decisions taken while a wrong revision was live are found by date, so those firms can be told
Diff too large for the LLM → no summary	<i>In preparation</i> too long	Summarize by section, or the content team writes it
Firm-scoping bug → cross-firm leak	Tests that try to read another firm's data; access logs	Firm from the session token; tables in each firm's own database
Backfill writes an old reading after an apply	Spot check	Writes carry the revision number; older ones are ignored
Engagement cannot be loaded, or our table disagrees with it	<code>error</code> count; spot check	Shown as <code>error</code> , never <i>Up to date</i> ; fixed on next open
Plan B: the engagement management system cannot share a transaction with our tables (assumption 4 fails) → a decision could be saved without us knowing	Spot check	The hook sends each change as a message after the save, retried until stored; the revision number makes late or repeated messages harmless

Tradeoffs accepted (what we chose over the alternative: gain; cost)

- **Our own table over changing the engagement system so versions are readable without loading:** that core system stays untouched; the table can disagree with the file (hence the spot check).
- **Computing pending when the list is opened over marking every affected engagement at publish:** a publish costs one row however many firms use the template; users learn of a new version when they look, not by notification.
- **Loading every existing engagement once (backfill) over waiting until each is opened:** no version stays unknown; ~150,000 one-minute loads, once.
- **Tables in each firm's database over one central database:** isolation and residency come from where the data lives; every firm's database needs a migration.
- **Same transaction as the user's decision over an event processed later:** a decision can never be missing from our tables; if our write fails, the user's apply/decline fails too.
- **One review for all pending versions over one decision per version:** one click in the common case; versions are cumulative, so declining v5 and later applying v6 brings v5's changes in (the review shows it).
- **LLM summaries over fixed rules per change type:** readable without knowing the template's structure; the LLM can be wrong, hence the ID check and human review.
- **One summary per version over one per jump (e.g. v3 → v6):** each is written and reviewed once for all firms; users read changes version by version, not the net effect (*changed again* helps).